אפיון מלא — דשבורד בוקר ומערכת תכנון חכמה

ליתאי ניהול חשבונאות | CRM | Vite + React + TypeScript + Supabase

---

רקע והבנת הבעיה

מה לא עובד עכשיו

1. המערכת דוחפת הכל לדדליין — "כדור שלג" של עומס ביום האחרון
2. אין הבדל בין משימה שאפשר להתחיל עכשיו למשימה שתלויה בלקוח
3. מורכבות מוגדרת ברמת לקוח — אבל לקוח יכול להיות ננו בשכר וכבד בהנהלת חשבונות
4. הדשבורד מציג רשימה שטוחה — אין היררכיה, אין תעדוף, אין כיוון
5. ימי שישי/שבת/חגים לא נלקחים בחשבון בתכנון

הפתרון — 3 שכבות

```
שכבה 1: הגדרות (פעם אחת)
  → עומס לכל תהליך + לקוח
  → מקור נתונים לכל תהליך
  → ימים חסומים

שכבה 2: תכנון גמיש (פעם בחודש בערך)
  → פריסת עבודה חכמה אחורה מהדדליין
  → התחשבות בעומס + ימים חסומים

שכבה 3: דשבורד בוקר (כל יום)
  → מה עושים היום — ברור ומדויק
  → אפשרות שינוי מהיר
```

חלק א' — שינויים בטבלת לקוחות

שדות קיימים שצריך לשנות שם

```
לפני:   "מורכבות" (מספר עובדים)
אחרי:   "מורכבות שכר" (מספר עובדים — כמו שיש)
```

שדות חדשים להוסיף לטבלת לקוחות

```sql
-- בסופהבייס: ALTER TABLE clients ADD COLUMN
complexity_bookkeeping  TEXT  -- 'low' | 'medium' | 'high'
                                -- עומס הנהלת חשבונות + מאזנים + רו"ח חודשיים
                                -- לא מספר — תגית, כי זה לא ניתן לכימות כמו עובדים
```

טבלת מיפוי עומס — לוגיקת חישוב

תחום	בסיס חישוב	כיצד
שכר	`payroll_employees` (קיים)	ליניארי: 1-3 = קל, 4-10 = בינוני, +11 = כבד
הנהלת חשבונות	`complexity_bookkeeping` (חדש)	ידני: קל/בינוני/כבד
מע"מ	`complexity_bookkeeping`	נגזר אוטומטית — זהה לה"ח
דוח שנתי + מאזן	`complexity_bookkeeping`	כבד יותר ב-50% מה"ח
פנסיות	`payroll_employees`	זהה לשכר

חלק ב׳ — שדות חדשים בטבלת תהליכים/תחומים

שדה: מקור נתונים

כל תהליך מקבל הגדרה של מאיפה מגיעים הנתונים.

```sql
-- בסופהבייס: ALTER TABLE task_types (או טבלת התהליכים הקיימת)
data_source  TEXT  -- 'self' | 'client' | 'mixed'
```

ערך	משמעות	השפעה על תכנון
`self`	יש לי גישה עצמאית לנתונים	נכנס לתכנון מיד, אפשר לפרוס
`client`	חייב לחכות שהלקוח ישלח	נשאר "ממתין" עד קבלה, לא נכנס לפריסה
`mixed`	חלק ממני, חלק מהלקוח	נכנס לתכנון חלקי + אינדיקטור "ממתין להשלמה"

דוגמאות לערכים

```
שכר                    → client  (תלוי בנתוני שעות/שינויים מהלקוח)
מע"מ                   → self    (יש גישה למערכת החשבונות)
הנהלת חשבונות      → mixed   (בנק = self, חשבוניות = client)
ביטוח לאומי         → self    (גישה למערכת)
דוח שנתי             → mixed
פנסיות               → client
```

חלק ג' — לוגיקת פריסת עומס חכמה

הרעיון המרכזי

```
במקום: כל המשימות → דדליין
במקום: מפרוסים אחורה מהדדליין לפי עומס + ימים זמינים
```

ימים חסומים (ברירת מחדל)

```typescript
```

```
// src/constants/blockedDays.ts

export const DEFAULT_BLOCKED_DAYS = {
  weekly: [5, 6],  // (6) ושבת (5) שישי — index 0 = ראשון

  // חגים קבועים — יש לעדכן בתחילת כל שנה
  holidays: [
    // 2026
    '2026-04-13', // ערב פסח
    '2026-04-14', // פסח
    '2026-04-20', // שביעי של פסח
    '2026-05-02', // ערב יום העצמאות
    '2026-05-03', // יום העצמאות
    '2026-06-01', // ערב שבועות
    '2026-06-02', // שבועות
    '2026-09-20', // ערב ראש השנה
    '2026-09-21', '2026-09-22', // ראש השנה
    '2026-09-29', // ערב יום כיפור
    '2026-09-30', // יום כיפור
    '2026-10-04', // ערב סוכות
    '2026-10-05', // סוכות
    '2026-10-11', // הושענא רבה
    '2026-10-12', // שמיני עצרת
  ]
};

// פונקציה: האם יום מסוים זמין לעבודה
export function isWorkDay(date: Date, override = false): boolean {
  if (override) return true;  // "אין ברירה" — מאפשר עבודה בחג

  const dayOfWeek = date.getDay();
  const dateStr = date.toISOString().split('T')[0];

  if (DEFAULT_BLOCKED_DAYS.weekly.includes(dayOfWeek)) return false;
  if (DEFAULT_BLOCKED_DAYS.holidays.includes(dateStr)) return false;

  return true;
}

// פונקציה: ספור ימי עבודה זמינים בין שני תאריכים
export function countWorkDays(from: Date, to: Date): number {
  let count = 0;
  const current = new Date(from);
```

```
  while (current <= to) {
    if (isWorkDay(current)) count++;
    current.setDate(current.getDate() + 1);
  }
  return count;
}
```

אלגוריתם הפריסה

```typescript
```

```
// src/utils/workloadSpread.ts

interface SpreadInput {
  deadline: Date;
  totalWorkload: number;       // יחידות עומס (מחושב לפי לקוח + תחום)
  dataSource: 'self' | 'client' | 'mixed';
  clientMaterialsExpected?: Date; // מתי הלקוח אמור לשלוח
}

interface SpreadResult {
  canStart: Date;               // מתי אפשר להתחיל
  recommendedStartDate: Date;   // מתי כדאי להתחיל
  dailyLoad: number;            // עומס מומלץ ליום
  warningDaysBuffer: number;    // כמה ימי חיץ נשארו
  status: 'ok' | 'tight' | 'overloaded';
}

export function calculateSpread(input: SpreadInput): SpreadResult {
  const today = new Date();
  const workDaysAvailable = countWorkDays(today, input.deadline);

  // אם תלוי בלקוח — מחשב מתאריך קבלת חומרים
  const effectiveStart = input.dataSource === 'client' && input.clientMaterialsExpec
    ? input.clientMaterialsExpected
    : today;

  const effectiveWorkDays = countWorkDays(effectiveStart, input.deadline);
  const dailyLoad = input.totalWorkload / effectiveWorkDays;

  // חיץ מומלץ: 20% מזמן העבודה
  const bufferDays = Math.floor(effectiveWorkDays * 0.2);

  return {
    canStart: effectiveStart,
    recommendedStartDate: subtractWorkDays(input.deadline,
      Math.ceil(input.totalWorkload / DAILY_CAPACITY) + bufferDays),
    dailyLoad,
    warningDaysBuffer: bufferDays,
    status: dailyLoad > DAILY_CAPACITY * 1.5 ? 'overloaded'
          : dailyLoad > DAILY_CAPACITY ? 'tight'
          : 'ok'
  };
}
```

```
// קיבולת יומית ממוצעת — ניתן לכוונון
const DAILY_CAPACITY = 8; // יחידות עומס
```

חלק ד' — מסך הבוקר (Morning Dashboard)

מה מוצג ובאיזה סדר

```
┌──────────────────────────────────────────────────────────┐
│ בוקר טוב, לנה · יום שלישי 14/04                          │
│ ממתינות [Z] · משימות פעילות [Y] · דדליינים קרובים [X]   │
├──────────────────────────────────────────────────────────┤
│                                                          │
│ ⏰ פס דדליינים                                           │
│  כל דדליין + כמה % הושלם + האם עדיין פתוח                │
│                                                          │
├──────────────────────────────────────────────────────────┤
│                                                          │
│ 🔴 עכשיו — חובה היום (N פריטים)                          │
│  [כרטיסיות עם פס התקדמות + צבע לקוח]                     │
│                                                          │
│ 🟡 אחר כך — רצוי היום (N פריטים)                         │
│  [כרטיסיות]                                              │
│                                                          │
│ ⏸ ממתין ללקוח (N פריטים)                                 │
│  [כרטיסיות אפורות — לא לוחצות]                           │
│                                                          │
│ ✅ הושלם היום (N פריטים)                                 │
│  [כרטיסיות מעומעמות עם קו חוצה]                          │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

"לוגיקת מיון ל"עכשיו" / "אחר כך

```typescript
```

```
function classifyTask(task: Task, today: Date): 'now' | 'later' | 'waiting' | 'done'
  if (task.status === 'done') return 'done';

  // ממתין — תלוי בלקוח ועדיין לא הגיעו חומרים
  if (task.dataSource === 'client' && !task.materialsReceived) return 'waiting';

  const daysUntilDeadline = differenceInDays(task.deadline, today);
  const spread = calculateSpread(task);

  // עכשיו — דדליין תוך 48 שעות, או שהמערכת תכננה להתחיל היום
  if (daysUntilDeadline <= 2) return 'now';
  if (isSameDay(spread.recommendedStartDate, today)) return 'now';

  // אחר כך — הכל השאר
  return 'later';
}
```

פעולות מהירות מהדשבורד (ללא פתיחת פאנל)

כל כרטיסייה בדשבורד מאפשרת:

```
[✓] מיידי, עם אנימציית דעיכה — סמן הושלם
[→] עם בקשת אישור קצרה — דחה ליום מחר
[⏸] הופך לאפור, יוצא מהתכנון — ממתין לחומרים
[↑↓] גרירה פשוטה — שנה עדיפות
```

חלק ה' — מסך תכנון גמיש

מתי נכנסים אליו

- לא כל יום — פעם בחודש בערך
- כשיש שינוי גדול (חג, לקוח שמאחר, דדליין שהוזז)
- כשהמערכת מזהירה "עומס חריג בשבוע הבא"

מה מוצג

```
┌──────────────────────────────────────────────────────┐
│ תכנון עבודה — אפריל 2026                              │
│ [← חודש קודם]  [30/04 - 14/04 ▼]  [חודש הבא →]       │
├──────────────────────────────────────────────────────┤
```

```
│                                                                │
│  לוח שנה גרפי — שורות = לקוחות, עמודות = ימים          │
│                                                                │
│            15/4   16/4   17/4   20/4   21/4                    │
│            א'      ו'      ג'      ב'      א'                 │
│                                   [חג]   [חג]                  │
│  ביטוח לאומי  ▓▓▓
```

```
  [בינוני] → נקודה בינונית צהובה
  [כבד]    → נקודה גדולה טרקוטה
```

אזהרת "כדור שלג" — מניעה מוקדמת

```
אם: ימי עבודה זמינים < עומס נדרש × 1.2
אז: הצג כרזה בדשבורד:

┌──────────────────────────────────────────────┐
│ ⚠️ עומס חריג ב-17/4 — 3 ימים עד הדדליין      │
│ מומלץ: התחל את [שם לקוח] כבר היום            │
│ [הצג פתרון] [התעלם]                          │
└──────────────────────────────────────────────┘
```

חלק ז' — שינויי Supabase הנדרשים

1. שדות חדשים בטבלת לקוחות

```sql
ALTER TABLE clients
  ADD COLUMN complexity_bookkeeping TEXT
    CHECK (complexity_bookkeeping IN ('low', 'medium', 'high'))
    DEFAULT 'medium';
```

2. שדות חדשים בטבלת תהליכים

```sql
ALTER TABLE task_types  -- (או שם הטבלה בפועל)
  ADD COLUMN data_source TEXT
    CHECK (data_source IN ('self', 'client', 'mixed'))
    DEFAULT 'self';
```

3. שדות חדשים בטבלת משימות

```sql
```

```sql
ALTER TABLE tasks
  ADD COLUMN materials_received    BOOLEAN DEFAULT false,
  ADD COLUMN materials_expected_date DATE,
  ADD COLUMN planned_work_date     DATE,     -- יום שתוכנן לבצע
  ADD COLUMN cognitive_load        INTEGER, -- מחושב, לא ידני
  ADD COLUMN override_blocked_day  BOOLEAN DEFAULT false; -- "אין ברירה"
```

4. טבלת חגים וימים חסומים

```sql
CREATE TABLE blocked_days (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  date        DATE NOT NULL UNIQUE,
  reason      TEXT,  -- 'holiday', 'eve', 'personal'
  is_override BOOLEAN DEFAULT false
);
```

חלק ח' — תוכנית יישום לקלוד קוד

⚠️ כלל על: אל תשבור מה שעובד

קרא את CLAUDE.md לפני כל שינוי.
שינויי עיצוב = בלבד className ו-style.
שינויי מבנה נתונים = הוסף עמודות, אל תמחק.

שלב 1 — שינויי בסיס נתונים (בלי לגעת ב-UI)

```
[ ] הוסף complexity_bookkeeping לטבלת clients
[ ] הוסף data_source לטבלת task_types
[ ] הוסף materials_received, planned_work_date לטבלת tasks
[ ] צור טבלת blocked_days 2026 עם חגי
[ ] וודא שכל הקוד הקיים עדיין עובד אחרי השינויים
```

שלב 2 — עמודה חדשה בטבלת לקוחות

```
[ ] הוסף עמודת "מורכבות הנה"ח" לטבלה הקיימת
[ ] "מורכבות שכר" → "מורכבות" שנה שם (רק תווית UI, לא שם עמודה ב-DB)
[ ] הוסף dropdown קל/בינוני/כבד לעמודה החדשה
[ ] בדוק שכל הפילטרים והמיונים הקיימים עדיין עובדים
```

data_source שלב 3 — מסך תהליכים: הוסף

```
[ ] הוסף לכל תהליך שדה "מקור נתונים" (גישה עצמאית / ממתין ללקוח / משולב)
[ ] הצג תגית צבעונית בכל שורת תהליך
[ ] אפשר עריכה ידנית
```

שלב 4 — לוגיקת חישוב עומס

```
[ ] צור src/utils/workloadSpread.ts
[ ] צור src/constants/blockedDays.ts עם חגי 2026
[ ] כתוב פונקציות: isWorkDay, countWorkDays, calculateSpread
[ ] כתוב טסטים בסיסיים לפונקציות
```

שלב 5 — דשבורד בוקר חדש

```
[ ] צור src/pages/MorningDashboard.tsx
[ ] ממש פס דדליינים עם % השלמה
[ ] ממש 4 אזורים: עכשיו / אחר כך / ממתין ללקוח / הושלם
[ ] ממש פעולות מהירות: הושלם / דחה / ממתין
[ ] ממש אנימציית דעיכה על השלמה
[ ] חבר לאלגוריתם הפריסה
```

שלב 6 — מסך תכנון גמיש

```
[ ] צור src/pages/FlexPlanning.tsx
[ ] ממש לוח שנה גמיש (לא שבוע קבוע)
[ ] ממש אזהרת עומס חריג
[ ] ממש כפתור "פרוס אוטומטית"
[ ] ממש שמירת תכנון שמזין דשבורד בוקר
```

סיכום — מה המערכת יודעת לעשות אחרי היישום

- ✅ מחשבת עומס לכל לקוח לפי תחום (לא עומס גורף)
- ✅ יודעת אילו תהליכים מחכים ללקוח ואילו לא
- ✅ לא מתכננת עבודה בשישי/שבת/חגים
- ✅ מזהירה מוקדם כשיש עומס חריג
- ✅ דשבורד בוקר אומר "התחילי כאן" — לא "הנה הכל"

- ✅ גמישות מלאה — כל תכנון ניתן לשינוי יומי
- ✅ ממתין ללקוח = לא מופיע בתכנון עד שמגיע